

Keyword Spotting (KWS) Final Project — Four-Phase Plan

This plan is derived entirely from your uploaded lecture slides, proposal, and MATLAB scripts. It satisfies the EEE598 final project requirements (runnable MATLAB code + IEEE-style report), uses only class material (Energy, ZCR, MFCC, k-NN/SVM), and can be completed within one week (~26–30 hours).

Phase 1: Dataset Setup & Preprocessing (Day 1–2, ~4 hours)

Goal:

Prepare a small, balanced subset of the **Google Speech Commands v2** dataset (e.g., "yes," "no," "stop," "go," and "unknown/silence"). Standardize all clips for frame-based analysis.

Procedure:

1. **Dataset Curation:**
2. Use ~1,000–1,500 clips per keyword for manageable runtime.
3. Split into training (70%), validation (15%), and testing (15%) sets.
4. **Preprocessing:**
5. Resample to **16 kHz**, normalize amplitude, and trim silence.
6. Adopt frame length **20–25 ms** with **10 ms hop** using a **Hamming window**.
7. **Outputs:**
8. Store processed clips in organized folders.
9. Export metadata table with labels and file paths (`metadata.csv`).

Verification:

- Plot normalized waveforms to confirm equal duration and amplitude range.
 - Verify dataset balance per class using a histogram or bar plot.
-

Phase 2: Feature Extraction (Energy, ZCR, MFCC) (Day 2–4, ~7 hours)

Goal:

Implement and verify a **frame-based feature pipeline** for short-time **energy**, **zero-crossing rate (ZCR)**, and **Mel-Frequency Cepstral Coefficients (MFCCs)** as covered in Class 7 slides.

Procedure:

1. **Frame & Windowing:**
2. Use your consolidated MATLAB code pattern for energy and ZCR.

3. For each frame:

```
E = sum((x(n).^2).*w);  
ZCR = sum(xwin(1:end-1).*xwin(2:end) < 0);
```

4. **MFCC Pipeline:**

5. Compute **STFT**, apply **Mel filterbank**, take **log**, then apply **DCT**.

6. Keep **12 MFCCs + 1 energy** and compute **Δ (delta)** and **$\Delta\Delta$ (acceleration)** features for temporal dynamics (total $\approx 39D$).

7. **Feature Packaging:**

8. Concatenate [E, ZCR, MFCCs] per clip (mean or pooled across frames).

9. Save all features in .mat files for later training.

10. **Visualization:**

11. Plot energy/ZCR over time (verify voiced/unvoiced distinctions).

12. Generate MFCC spectrograms to confirm expected patterns.

Verification:

- Check feature value ranges for non-zero, realistic magnitudes.
- Validate $\Delta/\Delta\Delta$ values are smaller than static cepstra (reflecting derivatives).

Phase 3: Classifier Implementation & Training (Day 4–6, ~6–7 hours)

Goal:

Train and compare **k-NN** and **SVM** classifiers using extracted features. Evaluate the effect of combining time-domain and spectral features.

Procedure:

1. **Model Training:**

2. **k-NN:** Use `fitcknn` with $k = 3, 5$ (Euclidean distance).

3. **SVM:** Use `fitcsvm` with linear and RBF kernels.

4. **Feature Variants:**

5. **Set A:** MFCC-only.

6. **Set B:** MFCC + Energy + ZCR.

7. **Validation:**

8. Use cross-validation (`cvpartition`) for hyperparameter tuning.

9. Track training accuracy, validation accuracy, and confusion matrices.

10. **Logging & Plotting:**

11. Record performance across feature sets.

12. Visualize confusion matrices and accuracy vs. k /kernel plots.

Verification:

- Ensure accuracy improves with added Energy/ZCR (as hypothesized).
- Confirm confusion matrices show high diagonal dominance.

Phase 4: Testing, Robustness & IEEE Report (Day 6–7, ~8–10 hours)

Goal:

Evaluate final models on unseen data and under moderate noise conditions. Summarize results in a concise 4–5 page IEEE-style report.

Procedure:

1. **Testing:**
2. Run the trained models on the **test set** (unseen speakers).
3. Compute **accuracy**, **precision**, and **confusion matrices**.
4. **Noise Robustness:**
5. Add controlled background noise to the test samples.
6. Compare performance drop between clean vs. noisy conditions.
7. **Result Analysis:**
8. Compare MFCC-only vs. combined features.
9. Discuss model trade-offs (accuracy vs. complexity).
10. **Report Writing:**
11. **Abstract:** Problem, dataset, pipeline, headline accuracy.
12. **Methods:** Frame analysis (20 ms, 10 ms hop, Hamming window), Energy/ZCR formulas, MFCC derivation, 39D feature composition.
13. **Experiments:** Data subset, model details, evaluation metrics.
14. **Results:** Accuracy tables, confusion matrices, noise impact.
15. **Conclusion:** Classical DSP techniques achieve robust KWS performance on small-vocabulary speech.

Expected Results:

- Overall accuracy: **75–90%**, depending on feature combination.
- Small drop under noise (~5–10%).
- Clear advantage for MFCC + Energy + ZCR over MFCC-only.

Deliverables Summary

Phase	Focus	Hours	Key Outputs
I	Dataset Setup & Preprocessing	4	<code>prepare_dataset.m</code> , <code>metadata.csv</code>
II	Feature Extraction (Energy/ZCR/MFCC)	7	<code>extract_features.m</code> , <code>.mat</code> feature files

Phase	Focus	Hours	Key Outputs
III	Classifier Training (k-NN/SVM)	7	<code>train_models.m</code> , accuracy & confusion matrices
IV	Testing, Robustness, Report	8-10	<code>evaluate_system.m</code> , IEEE-style report

File Structure Suggestion

```

/kws_project/
  prepare_dataset.m
  extract_features.m
  train_models.m
  evaluate_system.m
  /data/
  /features/
  /results/
  kws_report.pdf

```

Alignment with Class Material

- Frame-based analysis, short-time Energy/ZCR → from DSP & Frame Processing lectures.
- MFCC extraction pipeline (Mel-filterbank → log → DCT → $\Delta/\Delta\Delta$) → from Class 7 slides.
- Classifiers (k-NN, SVM) → from Acoustic Modeling & ML section.
- Performance metrics (accuracy, confusion matrix) → from proposal and class exercises.

This plan demonstrates full mastery of course topics, follows the 30-hour allocation in your proposal, and is executable within one week while satisfying all grading criteria.

Python Execution Guide — Explanation-Only (No Code)

Below expands each phase with **what** it should do, **why** it matters (per slides and proposal), **how** to verify it worked, and **what to record** for your report. This mirrors the exact parameters and methods from the uploaded materials.

Phase 1 — Dataset Setup & Preprocessing (Explanation)

What this phase does - Curates a small, balanced subset of Google Speech Commands v2 containing the target words (e.g., "yes," "no," "stop," "go," plus an unknown/silence class). - Standardizes all audio to the same sampling rate (16 kHz), loudness scale (normalization), and duration (~1 s clips from the dataset), so all downstream feature calculations are comparable. - Establishes splits (train/validation/test) so model selection does not leak into final evaluation.

Why it matters (course tie-in) - Consistent sampling and normalization ensure the **frame-based analysis** behaves as in the lectures (Hamming windows, fixed frame/hop). - Balanced classes prevent trivial classifiers from overfitting to frequent labels and make **confusion matrices** interpretable.

Parameters to fix (from slides/proposal) - Sampling rate: **16 kHz**. - Frames: **~20–25 ms** window length; **10 ms** hop; **Hamming** window. - Optional pre-emphasis: **0.97** before framing.

Quality checks - Plot a few normalized waveforms to ensure amplitudes are within $[-1, 1]$ and that files are not clipped or silent. - Count items per class; verify the intended balance and the split proportions.

Artifacts to save - A metadata table (file path, label, split). - A short note explaining any downsampling of the dataset (e.g., ~1,000–1,500 clips per class for runtime feasibility).

Phase 2 — Feature Extraction (Energy, ZCR, MFCC) (Explanation)

What this phase does - Converts each standardized waveform into frame-level features that capture the information highlighted in the course: - **Short-time Energy**: magnitude/voicing intensity per frame. - **Zero-Crossing Rate (ZCR)**: sign-change rate; relates to voicing/noisiness. - **MFCCs**: spectral envelope via **Mel filterbank** → **log** → **DCT**, using the first **12 cepstral coefficients** (low-frequency envelope) as taught. - **Dynamics**: add **deltas** and **double-deltas** to capture temporal change, and include overall **energy**, forming the **typical ~39-D feature vector** per frame.

Why it matters (course tie-in) - The **DCT** step decorrelates features, which is why simple models (including diagonal-covariance assumptions historically) work better on MFCCs. - Adding **energy** and **$\Delta/\Delta\Delta$** captures information the static cepstra do not (overall intensity and changes over time), a standard practice emphasized in the slides.

Parameters to fix (from slides/proposal) - Frame size: **~25 ms**; Hop: **10 ms**; **Hamming** window. - **12 MFCCs** retained; include **energy**, **Δ** , **$\Delta\Delta$** .

How to aggregate for keyword-level classification - Pool frame features into a single vector per clip (e.g., mean and standard deviation across frames) so k-NN/SVM can classify each one-second clip.

Quality checks - Visualize Energy/ZCR overlays with the waveform; energy should peak on vowels, ZCR higher in noisy/unvoiced regions. - Inspect MFCC matrices (time × coefficient) for stable ranges; **$\Delta/\Delta\Delta$** magnitudes should be smaller than static cepstra.

Artifacts to save - Per-clip feature files and a compact manifest noting which feature set each experiment uses (MFCC-only vs MFCC+Energy+ZCR).

Phase 3 — Classifier Implementation & Training (Explanation)

What this phase does - Trains **k-Nearest Neighbors (k-NN)** and **Support Vector Machine (SVM)** classifiers using the features above. - Compares two feature configurations required by the proposal: **MFCC-**

only vs MFCC + Energy + ZCR. - Uses the **validation set** to choose k (for k-NN) and kernel type (for SVM) before touching the test set.

Why it matters (course tie-in) - Slides discuss k-NN and parametric modeling as foundational classifiers for MFCC features; this phase demonstrates those concepts directly on your KWS task.

Parameters to fix (from slides/proposal) - Try small k values (e.g., 3 or 5) for k-NN. - Try linear and RBF kernels for SVM and select by validation accuracy.

Quality checks - Log validation accuracy for each model/feature configuration. - Plot **confusion matrices** for the best settings on validation to ensure classes are separable and errors are sensible (e.g., specific pairs occasionally confused).

Artifacts to save - A results table (model, feature set, chosen hyperparameters, validation accuracy). - Confusion matrix images for the chosen configurations.

Phase 4 — Testing, Robustness, and IEEE-Style Report (Explanation)

What this phase does - Freezes the best model (per validation) and evaluates it **once** on the **held-out test set** (unseen speakers). - Runs a **moderate-noise** robustness probe by adding background noise and measuring the accuracy change. - Produces the final **accuracy** and **confusion matrices**, then writes a concise **4-5 page IEEE-style report** summarizing methods and findings.

Why it matters (course tie-in) - The proposal requires accuracy and confusion-matrix analysis and suggests testing under moderate background noise to examine robustness and generalization.

Parameters to fix (from slides/proposal) - Report the main test accuracy and the noisy-test accuracy. - Include compact comparisons: MFCC-only vs MFCC+Energy+ZCR; k-NN vs SVM.

Quality checks - Ensure no validation or training files appear in the test set. - Sanity check per-class accuracies to confirm class balance and identify weak spots.

Artifacts to save - Final metrics table and confusion matrices (clean vs noisy). - IEEE-style report with: Abstract; Methods (framing, Energy/ZCR, MFCC, Δ/Δ); Experiments (dataset subset, models, splits); Results (accuracy, confusions, robustness); Conclusion (what worked and why, within the classical pipeline).

Success Criteria (for full credit)

- **Reproducible pipeline:** preprocessing → features → training → evaluation, with consistent parameters fixed in one place.
- **Comparative analysis:** MFCC-only vs MFCC+Energy+ZCR; k-NN vs SVM; clean vs moderate-noise.
- **Clear artifacts:** metadata, feature files, confusion matrices, final metrics, IEEE-style report.
- **Alignment with lectures:** frame sizes, Hamming window, pre-emphasis option, 12 MFCC + energy + Δ/Δ , simple classifiers, accuracy/confusion reporting.

Common Pitfalls to Avoid

- Mixing validation and test during model selection.
- Inconsistent sampling rates or window parameters between phases.
- Forgetting to include energy and $\Delta/\Delta\Delta$ when constructing the typical MFCC feature set.
- Reporting only overall accuracy without confusion matrices.